

Block Diffusion 학습 Loss 정식화

확산 모델 학습 loss (mask / graph 커널) + Discriminator · Classifier 학습 loss

Block-Diffusion-OD-Planning (v2 backbones)
자매 문서: BD_BLOCK_GUIDANCE_MATH.pdf, BD_DCBG_MATH.pdf

2026-07-21

Abstract

본 문서는 학습에 쓰이는 모든 loss 를 수식으로 정리한다: (1) block-diffusion 확산 모델 학습 loss — 흡수(mask) 커널의 SUBS 가중 CE 와 균일(graph) 커널의 D3PM ELBO(+보조 CE·연결성 정식화)—, (2) guidance 용 두 판별기의 학습 loss —IW discriminator(clean 부분경로 BCE)와 D-CBG classifier(노이즈 상태 BCE). 각 수식에 구현 위치(models_seq/bd_models.py, bd_trainer.py, train_bd_disc.py, train_dcbg_classifier.py)를 병기한다.

1 Canvas 와 loss mask (길이 처리)

경로 $[ori, v_1, \dots, v_L=dst]$ 를 canvas $x_0 = [dst, ori, v_1, \dots, v_L=dst, \langle END \rangle, \dots \mid PAD, \dots]$ 로 만든다 (build_canvas). 학습 대상 집합 $\mathcal{M}$ (loss mask) 은:

- 학습됨: 경로 본문 $v_1..v_L + dst$ 가 속한 블록의 $\langle END \rangle$ tail.
- 제외: dst 블록 이후의 **PAD** 블록 전체 (노이즈·loss 대상 아님).
- 조건 dropout 된 표본은 prefix $[dst, ori]$ 도 노이즈+학습(무조건 생성용): $\mathcal{M}[\neg cond, :pfx]=True$ (bd_models.py:435, 498).

블록별 시각 t_b 를 뽑아 토큰별 $t_{tok} = t_b$ (블록 내 공유)로 확장한다.

2 확산 학습 loss — mask (흡수/SUBS) 커널

블록별 $t_b \sim \mathcal{U}[10^{-3}, 1]$. 각 학습 위치 $i \in \mathcal{M}$ 는 확률 t_i 로 [MASK] 로 치환:

$$move_i = \mathbf{1}[u_i < t_i] \wedge \mathbf{1}[i \in \mathcal{M}], \quad x_t^i = \begin{cases} [\text{MASK}] & move_i \\ x_0^i & \text{else} \end{cases} \quad (\text{bd_models.py:437--438}). \quad (1)$$

SUBS 파라미터화: [MASK] 상태의 확률을 0 으로 두고 (logits[MASK] = $-\infty$) softmax, $\log p_\theta(x_0^i \mid x_t, x^{<b}) = [\log \text{softmax}(\text{logits}_i)]_{x_0^i}$ (:451--453). Loss 는 마스킹된 위치의 $1/t$ -가중 음의 로그가능도:

$$\mathcal{L}_{\text{mask}} = \frac{1}{|\mathcal{M}|} \sum_i move_i \cdot \left(-\frac{1}{t_i} \log p_\theta(x_0^i \mid x_t, x^{<b}) \right) \quad (\text{bd_models.py:456--458}). \quad (2)$$

$1/t$ 가중은 MDLM/SUBS 흡수 ELBO 의 연속시간 가중이며, 정규화 분모는 loss-mask 크기 $|\mathcal{M}|$ 이다. (bd3lms diffusion.py:819--891 대응.)

BD3-LM NELBO (식 8) 와의 대응

BD3-LM 의 블록 확산 NELBO 는

$$-\log p_\theta(\mathbf{x}) \leq \mathcal{L}_{\text{BD}} = \sum_{b=1}^B \mathbb{E}_{t \sim [0,1]} \mathbb{E}_q \frac{\alpha'_t}{1 - \alpha_t} \log p_\theta(\mathbf{x}^b \mid \mathbf{x}_t^b, x^{<b}). \quad (\text{BD3-LM 8})$$

위 $\mathcal{L}_{\text{mask}}$ 가 이 식을 그대로 구현함을 세 부분으로 확인한다.

(i) **Weight 일치.** 코드는 선형(log-linear) 스케줄을 쓴다: 시각 t 에서 마스킹 확률이 정확히 t ($\text{move}_i = \mathbf{1}[u_i < t_i]$, `bd_models.py:437`) 이므로 미마스킹 확률은 $\alpha_t = 1 - t$. 따라서

$$\alpha'_t = -1, \quad 1 - \alpha_t = t \implies \frac{\alpha'_t}{1 - \alpha_t} = -\frac{1}{t}, \quad (3)$$

즉 식 8 의 weight 가 우리의 $-1/t$ 와 정확히 일치한다(임의 스케줄 weight 의 선형 스케줄 특수화; cosine 등 다른 스케줄이면 형태가 달라진다).

(ii) **Token 분해는 정확(근사 아님).** SUBS 는 위치별 독립(mean-field) 복원이며 absorbing ELBO 에서는 마스킹된 위치만 기여하므로 블록 재구성 항이 토큰합으로 분해된다:

$$\log p_\theta(\mathbf{x}^b \mid \mathbf{x}_t^b, x^{<b}) = \sum_{i \in b} \mathbf{1}[\text{move}_i] \log p_\theta(x_0^i \mid x_t, x^{<b}). \quad (4)$$

그러므로 블록 항은 정확히 토큰별 $1/t$ -가중 CE 의 합이고(`(nll*move).sum()`, :456--458), 식 8 의 $\sum_b$ 는 block-diffusion attention (`get_block_diff_mask`) 하의 전체 학습위치 합으로 실현된다($x^{<b}$ 는 clean 문맥).

(iii) **정규화의 불편성.** `move`(마스킹 위치)로만 합하고 분모를 $|\mathcal{M}|$ (전체 학습가능 위치)로 두는 것은, $\mathbb{E}_q[\text{move}_i] = t$ 에 의해

$$\mathbb{E}_q \left[\sum_i \text{move}_i \cdot \frac{1}{t} (-\log p_\theta^i) \right] = \sum_{i \in \mathcal{M}} (-\log p_\theta^i) \quad (5)$$

이므로 $1/t$ 가중 $\times \frac{1}{|\mathcal{M}|}$ 가 토큰당 ELBO 의 불편(*unbiased*) 추정이 된다 — 분모가 $|\text{masked}|$ 가 아니라 $|\mathcal{M}|$ 인 이유다.

Remark 1. 이 대응은 **mask(흡수) 커널** 전용이다. `graph`(균일 CTMC) 커널은 식 8 이 아니라 아래 §의 이산 D3PM ELBO(KL + 보조 CE + 연결성 정칙화)로 별도 구현된다. 단, 이는 블록 내부 커널의 차이일 뿐이며, 블록 사이의 semi-AR 조건화($x^{<b}$)는 두 커널이 동일한 메커니즘으로 공유한다(다음 § 참고).

3 확산 학습 loss — graph (균일 CTMC/D3PM) 커널

$\bar{Q}_t$ = 누적 전이행렬(matrices), Q_t = 한 스텝 전이(Q). 블록별 정수 $t_b \in \{1, \dots, T\}$ (uniform/early/cosine 샘플링, `bd_models.py:479--491`). 확산 상태는 $q(x_t \mid x_0) = \text{Cat}(x_0 \bar{Q}_t)$ 로 학습 위치에만 노이즈(PAD clean):

$$x_t^i \sim \text{Cat}([\bar{Q}_{t_i}]_{\cdot, x_0^i}) \quad (:504--506), \quad \hat{x}_0 = \text{softmax}(\text{logits}_{V_0}) \quad (:513). \quad (6)$$

이전 블록 조건화($x^{<b}$)는 **mask 커널과 동일한 메커니즘**이다. 모델 입력을 $x_{\text{in}} = [x_t \parallel x_0]$ 로 두고(noised 현재 $\parallel$ clean 전체) block-diffusion attention 으로 넣는다(`get_block_diff_mask`, :508--511). 이 attention 하에서 블록 b 의 noised query 는 concat 된 clean x_0 half 의 이전 블록들에 attend 하므

로, $\hat{x}_0$ 및 예측 posterior 가 $x^{<b}$ 에 조건화된다 — 이것이 아래 $p_\theta(\cdot | x_t, x^{<b})$ 의 $x^{<b}$ 이다. 즉 graph 커널도 블록 사이는 semi-AR(clean 이전 블록 조건), 블록 내부만 균일 CTMC 확산이다. 참(true) posterior 와 예측(pred) posterior (D3PM):

$$\begin{aligned} q(x_{t-1}=k | x_t, x_0) &\propto [Q_t]_{k,x_t} [\bar{Q}_{t-1}]_{k,x_0} & (:517--519), \\ p_\theta(x_{t-1}=k | x_t, x^{<b}) &\propto [Q_t]_{k,x_t} [\bar{Q}_{t-1}^\top \hat{x}_0]_k & (:529--531). \end{aligned} \quad (7)$$

Remark 2 (True posterior 의 차원별 독립성 — 이전 블록과 무관). D3PM forward 는 차원별 독립이므로 ($q(x_t | x_0) = \prod_i q(x_t^i | x_0^i)$), x_0 를 알면 **true posterior** 도 완전히 토큰(차원)별로 분해되어 위치 i 의 posterior 는 오직 (x_t^i, x_0^i) 에만 의존한다 ($q(x_{t-1}^i | x_t, x_0) \propto [Q_t]_{\cdot, x_t^i} \odot [\bar{Q}_{t-1}]_{\cdot, x_0^i}$, :517--519). 즉 같은 블록의 다른 토큰과도, 이전 블록과도 독립이며 모델·attention 이 개입하지 않는다. 이전 블록 조건화 $x^{<b}$ 는 오직 **predicted posterior** 에만, 그것도 $\hat{x}_0$ 를 통해서 들어간다 ($\hat{x}_0$ 가 block-diffusion attention 으로 clean $x^{<b}$ 에 조건화, :529--531). 따라서 $\text{KL}(q \| p_\theta)$ 의 타깃 q 는 $x^{<b}$ -독립 고정분포이고, 블록 조건화는 확산 과정·true posterior 가 아니라 **denoiser**($\hat{x}_0$ 예측기)에 대한 모델링 선택일 뿐이다. 이런 의미에서 graph 커널 손실은 GDP(seq_models.py Restorer)를 그대로 따르며, 블록 확산이 추가한 것은 (i) 블록별 독립 시간 t_b 와 (ii) $\hat{x}_0$ 의 $x^{<b}$ 조건화, 이 둘뿐이다.

per-token D3PM ELBO 항은 두 posterior 의 KL(masked mean):

$$\mathcal{L}_{\text{KL}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \text{KL}(q(x_{t-1} | x_t, x_0^i) \| p_\theta(x_{t-1} | x_t, x^{<b})) \quad (:537--538). \quad (9)$$

보조 x_0 교차엔트로피와 연결성 정칙화(인접행렬 A 위 bilinear, 연속 실정점쌍):

$$\mathcal{L}_{\text{CE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} -\log[\hat{x}_0^i]_{x_0^i} \quad (:539--540), \quad (10)$$

$$\mathcal{L}_{\text{con}} = -\frac{100}{Z} \sum_{\ell} \text{pair}_{\ell} \left(\hat{x}_0^{\ell \top} A \log \hat{x}_0^{\ell+1} + \hat{x}_0^{\ell+1 \top} A \log \hat{x}_0^{\ell} \right) \quad (:545--552). \quad (11)$$

총 loss (ce-gating, bd_trainer.py:39--52): CE 가 임계 미만이면 KL 만, 아니면 가중합:

$$\mathcal{L}_{\text{graph}} = \begin{cases} \mathcal{L}_{\text{KL}}, & \mathcal{L}_{\text{CE}} < 60/16 \text{ 또는 } \lambda_{\text{ce}} = \lambda_{\text{con}} = 0, \\ \mathcal{L}_{\text{KL}} + \lambda_{\text{ce}} \mathcal{L}_{\text{CE}} + \lambda_{\text{con}} \mathcal{L}_{\text{con}}, & \text{그 외.} \end{cases} \quad (12)$$

(seq_models.py 의 Restorer.forward 수식을 블록-확산 attention 으로 이식.)

4 판별기 학습 loss (1) — IW discriminator (clean 부분경로)

IW disc 는 *clean* 부분경로에서 “실제 예외 데이터 vs negative”를 BCE 로 구분한다 (label smoothing $\epsilon=0.05$, train_bd_disc.py:130--134):

$$\mathcal{L}_{\text{disc}} = \mathbb{E} \left[-\tilde{y} \log \sigma(\ell_\phi) - (1 - \tilde{y}) \log (1 - \sigma(\ell_\phi)) \right], \quad \tilde{y} = \begin{cases} 1 - \epsilon & (\text{pos}) \\ \epsilon & (\text{neg}) \end{cases} \quad (13)$$

$\ell_\phi = \lambda \tanh(z_\phi / \lambda)$ (bounded logit, $\lambda=4$). 입력은 clean 부분경로 $[x^{<b} \| x_0] +$ 시나리오 인접성(시간 조건 없음). positive = except 경로(무작위 절단 make_partial); negative 3택:

-neg	negative 분포	최적해 $\sigma(\ell_\phi)$ 가 주는 비율
data	normal 실제 경로	$q_{\text{exc}}/(q_{\text{exc}}+q_{\text{norm}}) \Rightarrow D/(1-D)=q_{\text{exc}}/q_{\text{norm}}$
model	그 블록 모델 unconditional 생성	$\Rightarrow D/(1-D)=q_{\text{exc}}/p_\theta$ (guidance 가 요구하는 분포)
mix	50% normal + 50% model	위 둘의 혼합

Remark 3. Bayes-최적 판별자에서 $\sigma(\ell_\phi) = \frac{q}{q+p_{\text{neg}}}$ 이므로 $\frac{D}{1-D} = \frac{q}{p_{\text{neg}}}$. -neg model 이면 $p_{\text{neg}} = p_\theta$ 라 guidance 가중치 $w = q/p_\theta$ 를 정확히 학습한다 (자매 문서 BD_BLOCK_GUIDANCE_MATH.pdf 식 (3) · Lemma 1).

5 판별기 학습 loss (2) — D-CBG classifier (노이즈 상태)

D-CBG classifier 는 노이즈 상태 x_t 를 시간 t 로 조건화하여 같은 BCE 로 학습한다 (train_dcbg_classifier.py:114--119)

$$\mathcal{L}_{\text{clf}} = \mathbb{E} \left[-\tilde{y} \log \sigma(z_\phi(x_t, t)) - (1 - \tilde{y}) \log (1 - \sigma(z_\phi(x_t, t))) \right] \quad (14)$$

positive/negative 를 생성 시점과 동일한 **forward** 로 손상시켜 입력한다:

- mask: corrupt_mask — j 개 확정 블록 + 현재 블록 rate t 마스킹 (dcbg_plugin.py:107).
- graph: corrupt_graph — 현재 블록을 $q(x_t | x_0)$ 로 정수 t 노이즈 ($:132$).

따라서 z_ϕ 는 전 노이즈레벨에 걸친 $p_\phi(y | x_t, t)$ 를 학습한다 — IW disc (§4) 와의 유일한 차이는 이 노이즈-상태 + 시간 입력이며, 아키텍처 (DCBGClassifierAdj: 동일 GraphSAGE + edge 피쳐 + bounded logit) · 데이터 · negative pool 은 통제된다.

6 수식 $\leftrightarrow$ 코드 대응표

수식	코드 위치
canvas · loss mask (길이 처리)	bd_models.py: build_canvas, :435, :498
mask SUBS $1/t$ 가중 CE $\mathcal{L}_{\text{mask}}$	bd_models.py:437--464
graph $q(x_t x_0)$ 노이즈	bd_models.py:504--506
true/pred posterior (D3PM)	bd_models.py:517--531
$\mathcal{L}_{\text{KL}}, \mathcal{L}_{\text{CE}}, \mathcal{L}_{\text{con}}$	bd_models.py:537--552
ce-gating 총합 $\mathcal{L}_{\text{graph}}$	bd_trainer.py:39--52
IW disc BCE + label smoothing	train_bd_disc.py:130--134
negative 3택 (data/mix/model)	train_bd_disc.py:94--128
D-CBG classifier BCE (노이즈 입력)	train_dcbg_classifier.py:114--119
생성-정합 corruption	dcbg_plugin.py:107 / :132